

Amortised analysis

CS3230 · Lecture 6

Demo course material, written for OpenPr

Why amortise?

- Some operations are expensive, but rarely
- Worst case per operation is too pessimistic

Amortised cost

- The average cost per operation over a worst-case sequence

Not an average over random inputs

Aggregate method

- Total cost of n operations, divided by n
- Example: binary counter increments

Binary counter

- Bit i flips every 2^i increments
- Total flips $< 2n$
- Amortised $O(1)$ per increment

Stack with multipop 1

- Each element is popped at most once
- n operations cost $O(n)$ in total

Stack with multipop 2

- Each element is popped at most once
- n operations cost $O(n)$ in total

Stack with multipop 3

- Each element is popped at most once
- n operations cost $O(n)$ in total

Stack with multipop 4

- Each element is popped at most once
- n operations cost $O(n)$ in total

Accounting method

- Charge each operation an amortised cost
- Store the extra as credit on the data
- Expensive operations spend the credit

Credit invariant

- Credit must never go negative
- Prove it holds after every operation

Potential method

- $\Phi(D)$ maps a state to a number
- Amortised = actual + change in Φ

Choosing Phi

- Phi starts at 0 and never goes negative
- Make Phi rise when cheap ops build up future work

Example: dynamic array

- Double the capacity when full
- Copying costs n , but only after n cheap appends

Dynamic array, accounting

- Charge 3 per append
- 1 pays for the append, 2 are saved for future copies

$O(1)$ amortised per append

Summary

- Aggregate, accounting, potential
- All give the same answer when done right